Вспомогательные алгоритмы

Процесс решения сложной задачи часть сводится к решению нескольких более простых подзадач. Соответственно процесс разработки сложного алгоритма разбивается на этапы составления отдельных алгоритмов, которые называются ***вспомогательными***. Каждый вспомогательный алгоритм описывает решение какой-либо подзадачи.

При разработке сложных алгоритмов используется метод последовательной детализации, который состоит в следующем.Сначала алгоритм формулируется в крупных блоках, которые еще нельзя перевести на язык операторов и записываются как вызовы вспомогательных алгоритмов. Затем происходит детализации, и все вспомогательные алгоритмы подробно расписываются для перевода на язык команд. Вспомогательные алгоритмы на языке программирования реализуются через подпрограммы.

Программы, которые неразделимы на отдельные структурные элементы, называются ***монолитными***. Большие монолитные программы сложны для разработки, отладки и сопровождения. Минимальным автономным элементом монолитной программы является оператор.

Естественно стремление разбить программу на более крупные, чем оператор, компоненты. Роль таких компонентов выполняют подпрограммы. Программы, содержащие подпрограммы, называются ***модульными***. Подпрограммы, в свою очередь, могут вызывать подпрограммы более низкого уровня и т.д. Таким образом, каждая модульная программа имеет иерархическую структуру.

Использование аппарата подпрограмм позволяет сократить объем и улучшить структуру программы с точки зрения наглядности и читаемости, уменьшить вероятность ошибок и облегчить процесс отладки программы. Разложение программы на взаимосвязанные, но замкнутые и логически завершенные компоненты дает возможность выполнять разработку отдельных подпрограмм разным программистам и более или менее независимо друг от друга. Кроме того, подпрограмма может быть рассмотрена как отдельная единица (со своими входными и выходными данными), что позволяет использовать ее в общем иерархическом подходе при конструировании алгоритма и программы по принципам нисходящего проектирования.

В языке C++ подпрограммы реализуются в виде функций и void функций. Существуют встроенные (стандартные) функции, которые являются частью языка и могут вызываться по имени без предварительного описания.

Программист имеет возможность определять свои собственные функции. Это можно делать либо в том же файле, в котором хранится основная часть программы, либо в отдельном файле, с тем чтобы эту функцию можно было использовать и в других программах. И в том и в другом случае определение будет одним и тем же, но пока предположим, что функция определяется в том же файле, в котором содержится функция main программы.

Описание функции состоит из двух частей, которые называются *прототипом функции* (function prototype) и *определением функции* (function

definition). **Прототип функции** описывает, как эта функция должна вызываться. В C++ перед вызовом функции в коде должно быть помешено либо полное определение функции, либо ее прототип.

Прототип функции предоставляет всю информацию, необходимую для того, чтобы вызвать эту функцию. В коде прототип должен располагаться до вызова функции, если только она не была определена до этого. Обычно прототипы функций размещают перед функцией main программы.

В определении функции описано, как именно эта функция должна вычислять возвращаемое значение. Если представить себе функцию как маленькую программу внутри вашей большой программы, то определение функции — это ее код. Синтаксис определения функции очень похож на синтаксис основной части программы. Определение функции состоит из *заголовка функции,* вслед за которым следует *тело функции.* Заголовок функции записывается гак же, как и ее прототип, но в конце заголовка, в отличие от прототипа, *не* ставится точка с запятой. Это приводит к тому, что заголовок повторяется, но в этом нет ничего страшного.

Хотя прототип функции и сообщает всю информацию, необходимую при ее вызове, он не сообщает, какое именно значение эта функция возвращает. Возвращаемое функцией значение определяется инструкциями *тела функции.* **Тело функции** следует после ее заголовка и завершает определение функции, состоящее из объявлений и выполняемых инструкций, заключенных в фигурные скобки. Таким образом, тело функции выглядит точно так же, как и та часть программы, которая следует после слова main. При вызове функции вместо ее формальных параметров подставляются реальные аргументы, после чего выполняются инструкции тела этой функции. Возвращаемое функцией значение определяется в момент вычисления инструкции return. Инструкция return состоит из ключевого слова return, за которым следует выражение.

Чтобы лучше разобраться в использовании функций, запомните три положения..

- Определение функции похоже на небольшую программу, а ее вызов подобен запуску этой "маленькой" программы.
- В функции вместо ввода, поступающего в программу с помощью потока cin, используются формальные параметры. Аргументы функции (**фактические параметры**), которые подставляются вместо формальных параметров, служат для нее входной информацией.
- Функция ничего не выводит на экран, однако возвращает некоторую информацию вызвавшей ее программе. Для этого она использует инструкцию return..

Размещение определения функции

Мы обсудили, где, как правило, размешаются прототипы и определения функций. В обычной ситуации указанное место является для них самым лучшим. Однако компилятор будет воспринимать и такие программы, в которых эти элементы кода размешаются в некоторых других местах. Сформулируем эти правила более точно: до вызова каждой функции компилятор должен встретить либо ее прототип, либо определение. Например, если в программе все определения

функций разместить перед основной частью программы, то прототипы функций уже не нужны. Знание этого общего правила поможет понять программы на C++, приведенные в других книгах, но все же лучше придерживаться образца, по которому написаны программы, представленные в этих работах. Используемого нами стиля придерживается большая часть программистов, которые работают на C++.

Процедурная абстракция

Применительно к определению функции принцип процедурной абстракции означает, что функцию нужно писать так, чтобы ее можно было использовать подобно *черному ящику*. Это значит, что программисту, который захочет использовать эту функцию, не нужно будет заглядывать в ее тело, чтобы понять, как она работает. Прототип функции и прилагающийся к ней комментарий должны предоставлять всю необходимую для использования этой функции информацию. Чтобы определение функции удовлетворяло этому требованию, нужно строго придерживаться приведенных ниже правил.

Как писать определение функции в виде черного ящика

- Комментарии к прототипу должны сообщать программисту все, что необходимо знать об аргументах функции, и описывать значение, возвращаемое функцией при ее вызове с этими аргументами.
- Все переменные, используемые в теле функции, должны быть объявлены в теле функции (формальные параметры объявлять не нужно, потому что они перечислены в ее прототипе).

Принцип процедурной абстракции гласит, что функция должна быть оформлена в виде самодостаточного модуля, который разрабатывается отдельно от остальной программы. При работе над большими программными проектами функции могут разрабатываться разными программистами, которые должны подбирать для формальных параметров имена, наиболее подходящие по смыслу. Аргументам (фактическим параметрам), которые подставляются вместо формальных параметров (и зачастую не тем программистом, который писал определение функции), тоже следует давать имена со смыслом. При этом может оказаться, что некоторые аргументы имеют одинаковые имена с формальными параметрами. Это был бы идеальный вариант. Однако независимо от того, какие имена выбраны для переменных, которые будут использованы в качестве аргументов, эти имена не перепутаются с именами формальных параметров. Ведь функции используют только значения аргументов, не обращая внимания на их имена.

Функции

Теория

Функция – это подпрограмма, вычисляющая и возвращающая некоторое значение.

Функция пользователя используется только тогда, когда требуется вычислить единственное значение. Функция возвращает результат в точку своего вызова. Поэтому функция является частью какого-нибудь выражения.

Синтаксис функции, которая возвращает значение:

Прототип функции:

Возвратаемый_тип Имя_функции {Список_параметров); Комментарии_к_прототипу

Определение функции:

```
//Следующая строка представляет собой заголовок функции
Возвращаемый_тип Имя_функции(Список_параметров)
 {  // Начало тела функции
      Объявление_1
      Объявление_2
      Объявление_N
      Исполняемая_инструкция_1      //  Одна из  этих
      Исполняемая_инструкция_2     // инструкций
      . . .                          // должна  быть
      Исполняемая_инструкция_N // инструкцией return
 } // Конец  тела функции
```

Поскольку функция не возвращает значения, пока не выполнит инструкцию return, в ее теле должна находиться по крайней мере одна такая инструкция (в теле функции может находиться и несколько инструкций return).

Для определения функции допустим любой образец размещения с помощью пробелов и символов новой строки. Однако лучше придерживаться тех же правил использования отступов от начала строки и размещения элементов кода, которые приняты для основной части программы. В частности открывающая ({) и закрывающая (}) скобки, которыми обозначаются границы тела функции, размещаются на отдельных строках. Тем самым тело функции отделяется от остального кода.

Данные, описанные в подпрограмме, действительны только в пределах данной пользовательской подпрограммы. Они называются **локальные параметры**. Данные, описанные в основной программе, называются глобальными и их можно так же использовать в подпрограмме. Если имя глобальной переменной совпадает с именем локальной, внутри подпрограммы эта переменная интерпретируется как локальная.

Параметры функции позволяют вычислять функцию с различными начальными данными. Параметры указываются в заголовке функции и называются формальными параметрами. По сути, они являются переменными, локальными для определения данной функции; их можно рассматривать как локальные переменные, которые объявляются при определении функции.

Описание формальных параметров имеет вид: *тип имя параметра*. Описание параметров разделяются запятой.

При вызове функции список формальных параметров заменяется фактическими параметрами. Между формальными и фактическими параметрами должны выполняться следующие правила соответствия:

- по количеству (количество формальных и фактических параметров одинаково);

- по последовательности (первому формальному параметру соответствует первый фактический параметр, второй – второму и т.д.);
- по типам (типы соответствующих формальных и фактических параметров должны совпадать).

Фактические параметры-аргументы могут бать выражениями соответствующего типа.

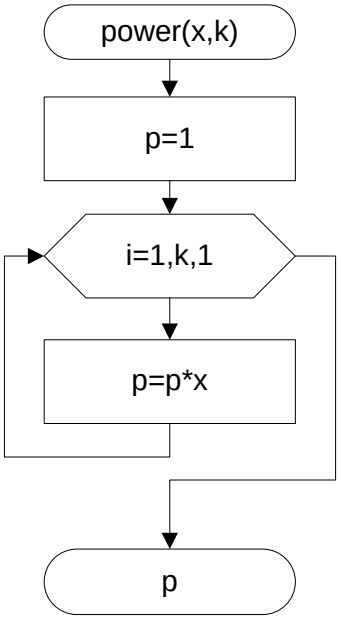
При разработке блок-схемы вспомогательный алгоритм разрабатывается как отдельная блок-схема, но в блоке «Начало» указывается заголовок подпрограммы.

Примеры

Пример 1.

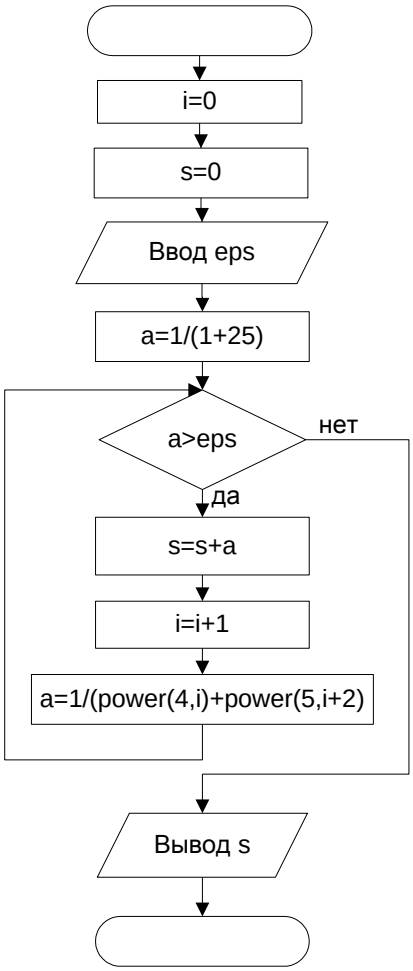
Вычислить с заданной точностью ε. $\sum_{i=0}^{\infty} \frac{1}{4^i + 5^{i+2}}$

Этот пример рассматривался в предыдущей лабораторной работе. Возведение в степень в нем выполнялось с помощью функции pow, но результат функции – это вещественное число, хотя в степень возводятся целые числа и результат тоже будет целым. В этом случае модно создать функцию возводящую целые числа в целую степень.




```
int power(int x, int k)
{
 int p=1;
 for(int i=1;i<=k;i++)
        p*=x;
 return p;
}
```

Исходные данные: точность eps вещественный тип, член последовательности a – вещественный тип.
Результат: сумма S – вещественный тип.
Тестовый пример: при eps=10^{-4}, S=0.0097.




```
# include <iostream>
# include <conio.h>
using namespace std;
int power(int x, int k);
void main()
{ int i=0;
  double eps,a,s=0;
  cout<<"eps="; cin>>eps;
   a=1.0/(1+25);

  while (a>eps)

        {s=s+a;

         i++;

         a=1.0/
(power(4,i)+power(5,i+2));
        }
  cout<<"s= "<<s<<endl;
  _getch();
}


int power(int x, int k)
{
 int p=1;
 for(int i=1;i<=k;i++)
         p*=x;
 return p;
}
```

Пример 2. Даны натуральные числа n, m, целые числа $a_1, \ldots, a_n$, $b_1, \ldots, b_m$, $c_1, \ldots, c_{10}$. Получить $w=x^2+y^2+z^2$, где $x=\min(a_1, \ldots, a_n)$, $y=\min(b_1, \ldots, b_m)$, $z=\min(c_1, \ldots, c_{10})$.

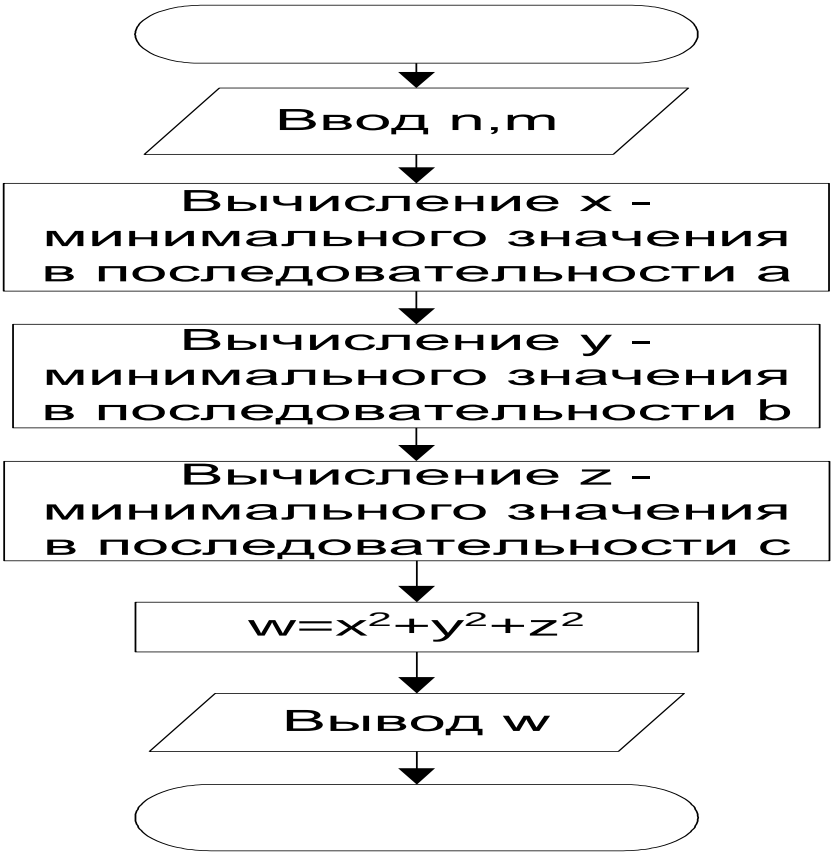
Исходные данные: n, m – целого типа, элемент последовательности a – целого типа, элемент последовательности b целого типа, элемент последовательности c– целого типа,

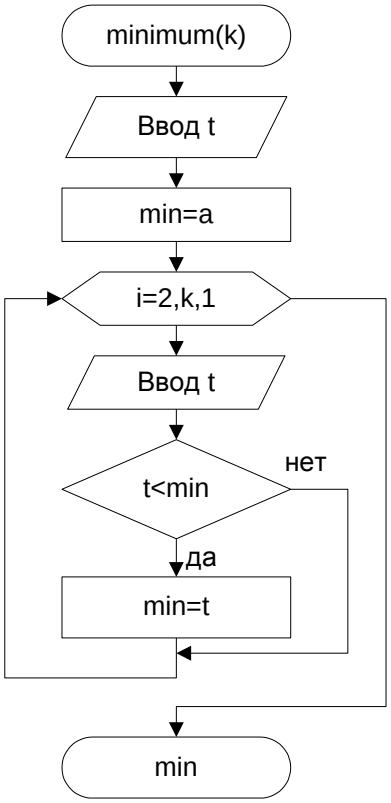
Результат: w целого типа

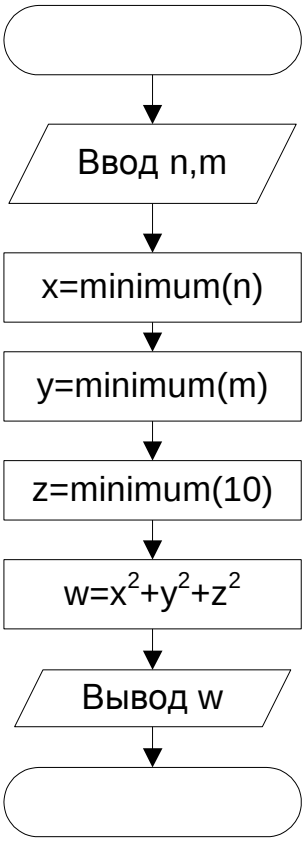
Приведен пример блок-схемы вычисления функции нахождения минимального значения.

Тестовый пример:

при n=7, m=8, последовательность a:
5, 2, 7, 0, 6, 1, 4; последовательность b:
-1, -3, -5, -3, -6, -2, -2, -7; последовательность c
-2, 4, 6, -8, 3, 5, -3, -5, -2, 1 w=65.


Ввод n,m
Вычисление x – минимального значения в последовательности a
Вычисление y – минимального значения в последовательности b
Вычисление z – минимального значения в последовательности c
w=x²+y²+z²
Вывод w


minimum(k)
Ввод t
min=a
i=2,k,1
Ввод t
t<min
нет
да
min=t
min

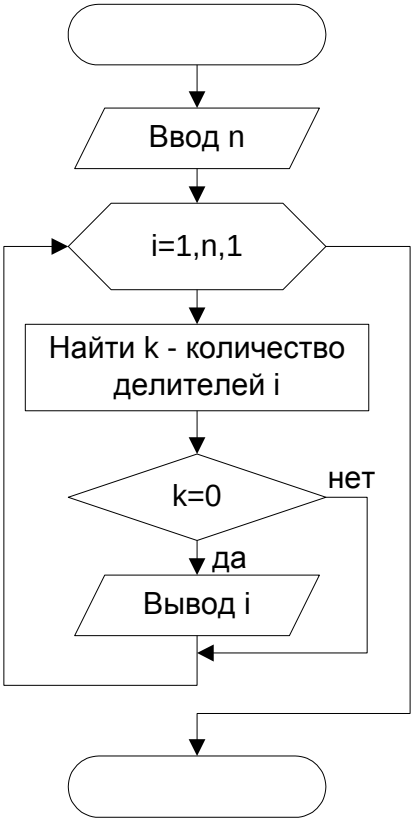

Ввод n,m
x=minimum(n)
y=minimum(m)
z=minimum(10)
w=x²+y²+z²
Вывод w

Пример 3.

Составить программу нахождения всех простых чисел не превосходящих n.

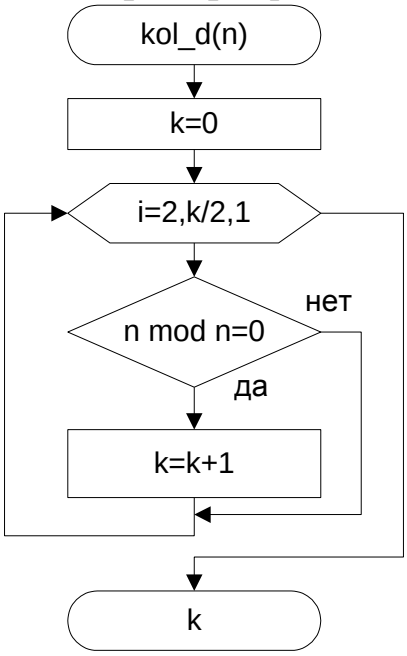
Исходные данные: n – целый тип.

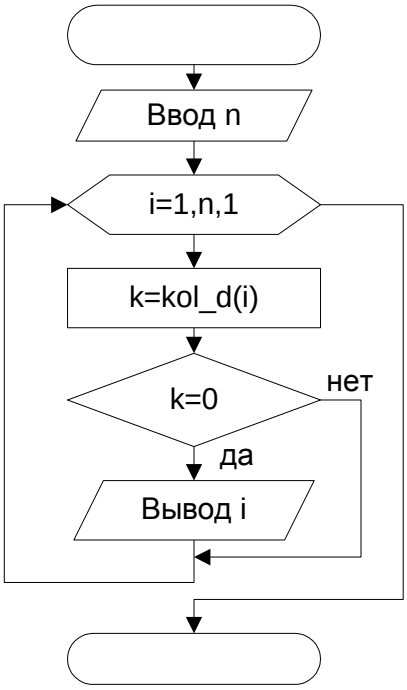
Результат: вывод i числа, количество делителей у которого равно 0.



При построении укрупненной блок схемы не выявлены повторяющиеся задачи, но подсчет количества делителей у числа можно рассматривать как самостоятельную задачу. Для этой задачи можно написать отдельную функцию.

Тестовый пример: при n=15, вывод 1, 2, 3, 5, 7, 11, 13.





Пример 4.

Написать программу, которая проверяет, являются ли во введенном четырехзначном числе вида $\overline{abcd}$, все цифры разные.

Исходные данные: четырехзначное число m.

Результаты: цифры числа m - a,b,c,d целый тип,

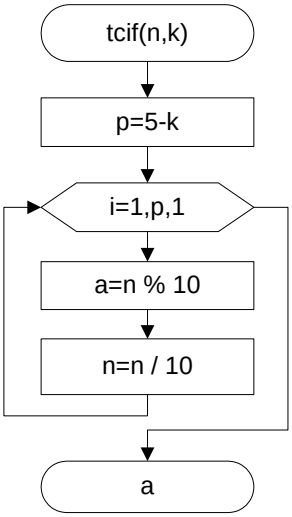
k=0, если есть совпадение цифр, k=1 если совпадения цифр нет.

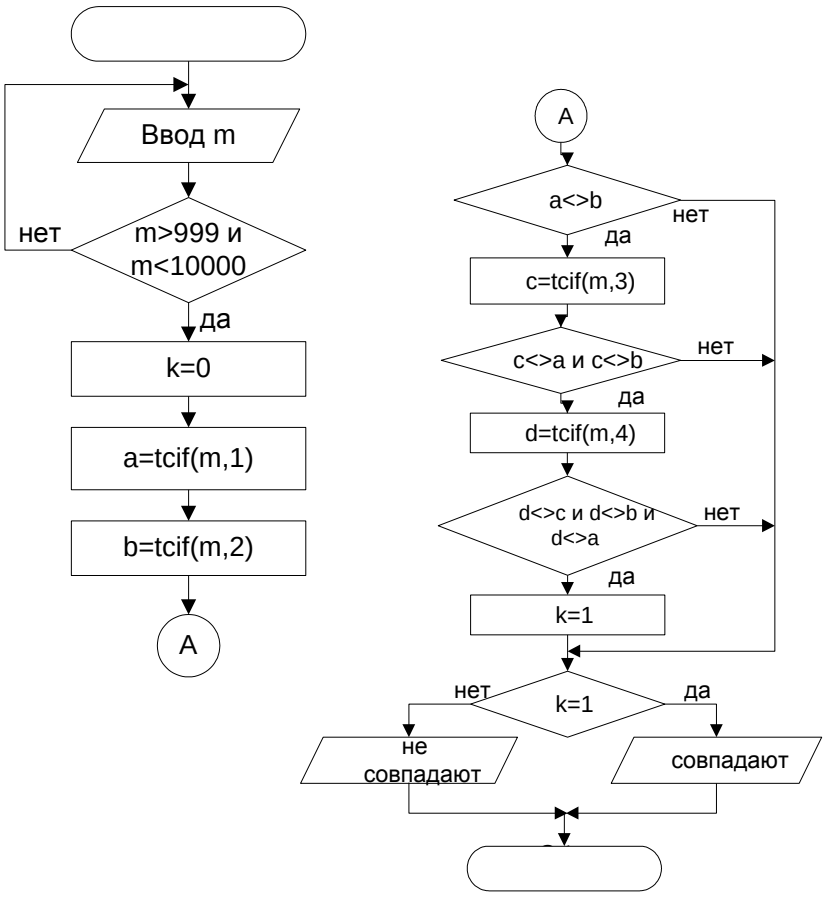
Следует создать функцию, которая выделяет заданную цифру из данного числа. Эта функция работает только с 4-значным числом, поэтому в начале программу следует проверить количество цифр в числе.

Тестовый пример:

при 2467 – все цифры разные;

при 1233 –цифры совпадают.



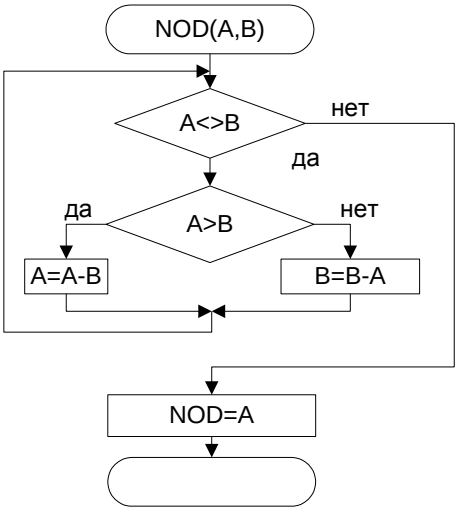


Пример 5.

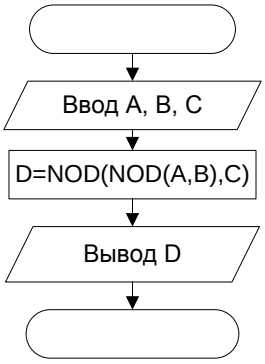
Найти наибольший общий делитель 3 чисел.

Исходные данные: A, B, C – целого типа.

Результат: наибольший общий делитель чисел A, B, C – D целого типа.



Нахождение наибольшего общего делителя (НОД) двух чисел выполняется в функции NOD. Сначала находится НОД для чисел A, B, а затем для полученного значения и D.



Процедуры (void-функции)

Теория

В C++ подзадачи реализуются в виде функций. Функции, которые рассматривались выше, всегда возвращают одно значение. Однако бывают и другие виды подзадач, например, когда требуется возвратить из функции несколько значений либо не возвращать вовсе ничего. Функции, которые не возвращают ни одного значения, называются процедурами (в C - void-**функциями).** Например, одной из типичных подзадач программы является вывод результатов вычислений. В этой подзадаче осуществляется вывод на экран, но она не вычисляет никаких значений, которые бы использовались в остальной части программы. Такую разновидность подзадач можно реализовать в виде void-функций.

Процедура (void-функция) – это независимая именованная часть программы, которую после однократного описания можно многократно вызывать по имени из последующих частей программы для выполнения определенных действий.

В C++ эти void-функции определяются аналогично функциям, возвращающим значение. Между определением void-функции и определениями функций, есть только два различия. Одно из них заключается в том, что вместо указания типа возвращаемого значения в определении void-функции используется ключевое слово void. Тем самым компилятору сообщается, что эта функция не возвращает никаких значений. Само имя void говорит: "эта функция не возвращает никаких значений". Второе различие состоит в том, что в инструкции return не содержится выражения,

по которому должно вычисляться возвращаемое значение, потому что никакого возвращаемого значения не существует.

Синтаксис определения void-функции

Прототип void-функции:

void *Имя_функции{Список_параметров)*; *//Комментарий_к_прототипу*

Определение void-функции:

```
void Имя_функции(Список_параметров)   //Заголовок функции
{      //Начало тела  функции
Объявление_1                //  Объявления можно чередовать
Объявление_2                //  с исполняемыми инструкциями
Объявление_N
Выполняемая инструкция_1   //  Можно  включить   одну или
Выполняемая инструкция_2  //   несколько   (необязательных)
Выполняемая инструкция_3  //  инструкций  return
Выполняемая инструкция_N
}     //Конец тела  функции
```

Вызов процедуры (void-функции) является выполняемой инструкцией. Она состоит из имени функции и списка фактических параметров отделенных друг от друга запятыми и заключенными в круглыми скобками. Иногда нужны функции без аргументов, что вполне корректно. В этом случае в прототипе функции список формальных параметров просто отсутствует и при ее вызове не передается ни одного аргумента.

Процедуры имеют более широкий спектр применения, чем функции:

- с помощью процедур можно одновременно вычислять несколько значений;
- в процедурах можно изменять входные параметры;
- результатом процедур не обязательно является вычисление значений, это может быть выполнение каких-нибудь действий, например ввод или вывод данных.

В процедурах (void-функциях), как и в функциях, возвращающих значение, могут присутствовать инструкции return. Если функция возвращает значение, оно определяется инструкцией return. В void-функциях эта инструкция просто завершает вызов функции, т.е.означает досрочный выход из функции.

При вызове функции ее аргументы подставляются вместо формальных параметров из определения этой функции; иными словами, аргументы "подключаются" к формальным параметрам. Есть несколько механизмов реализации такого процесса подстановки. Механизм, использовавшийся в предыдущем разделе, называется механизмом *передачи параметров по значению* (call-by-value). Второй важный механизм подстановки аргументов известен под названием *передачи параметров по ссылке* (call-by-reference).

При выполнении некоторых подзадач оказывается, что механизма передачи параметров по значению недостаточно. Например, довольно распространенной задачей является вычисление в функции одной или нескольких величин.

Аргументы функции, которые использовались до сих пор, могут быть переменными, но вместо формального параметра вызова по значению подставляется только *значение* аргумента. Для функции вычисляющей несколько значений нужно, чтобы вместо вычисляемого формального параметра подставлялась сама *перемен-*

ная, а не только ее значение. Механизм передачи параметров по ссылке работает следующим образом. Для формального параметра, передаваемого по ссылке, соответствующий аргумент вызова функции должен быть переменной, и эта переменная подставляется в тело функции вместо формального параметра. Все происходит так, как если бы переменная-аргумент (а не ее значение!) буквально копировалась в тело определения функции вместо формального параметра. После этого при выполнении тела функции значение переменной-аргумента может изменяться.

Чтобы компилятор мог отличить параметр, передаваемый по ссылке, от параметра, передаваемого по значению, необходимо явно указать способ передачи по ссылке. Для этого используется знак амнерсанда (&), который располагается после имени типа формального параметра, передаваемого по ссылке, как в прототипе функции, так и в заголовке ее определения.

В блок-схеме программы блок-схема процедуры, как и блок-схема функции, изображается отдельно. В блок-схеме основной программы оператор вызова подпрограммы изображается в виде блока:

Примеры

Пример 6.

Дана дробь в виде $\frac{A}{B}$ (A, B – натуральные числа). Представить эту дробь в виде несократимой дроби.

Исходные данные: A – числитель, целый тип и B – знаменатель, целый тип.

Результат: A – числитель после сокращения, B – знаменатель после сокращения.

Для сокращения дроби надо найти наибольший общий делитель (НОД) для числителя и знаменателя, а затем разделить числитель и знаменатель на это число.

Сокращение дроби запишем в отдельной viod-функции SOKR. Эта viod-функция в дальнейшем может быть использована в других задачах.

Данную задачу можно решить только с использованием viod-функции, т.к. в результате этой подпрограммы изменяются входные параметры.

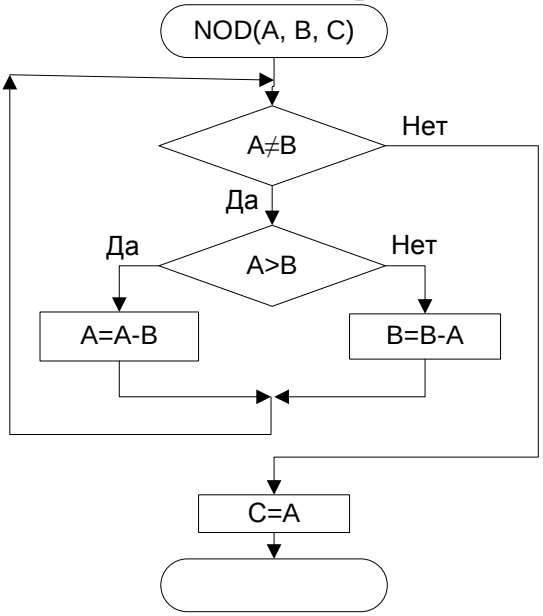
Обратите внимание, что функция NOD, которая была рассмотрена в предыдущей лабораторной работе, в данном примере представлена как viod-функция.

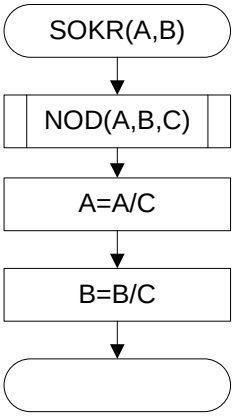
Любая функция может быть представлена как void-функция. В этом случае, полученный результат является еще одним параметром void-функции, причем параметром, передаваемым по ссылке.

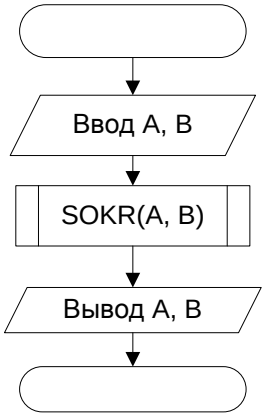
В данном примере используется две void-функции: void-функция NOD, для нахождения наибольшего общего делителя и void-функция SOKR для сокращения числителя и знаменателя. Void-функция SOKR вызывает процедуру NOD, поэтому в описании прототипов void-функция NOD должна предшествовать void-функции SOKR

Тестовый пример:

При A=7, B=56, после сокращения получаем A=1, B=8.








```
# include <iostream>
# include <conio.h>
using namespace std;
//Нахождение наибольшего общего делителя - c
void nod(int a, int b, int& c);
// сокращение дроби a/b
void socr(int& a, int& b);
void main()
{int a,b;
 cout<<"a="; cin>>a;
 cout<<"b="; cin>>b;
 socr(a,b);
 cout<<"a= "<<a<<" b= "<<b<<endl;
 _getch();
}
void nod(int a, int b, int& c)
{while (a!=b)
    {if (a>b)
            a=a-b;
    else b=b-a;
    }
    c=a;
}
void socr(int& a, int& b)
{int c;
    nod(a,b,c);
    a=a/c;
    b=b/c;
}
```

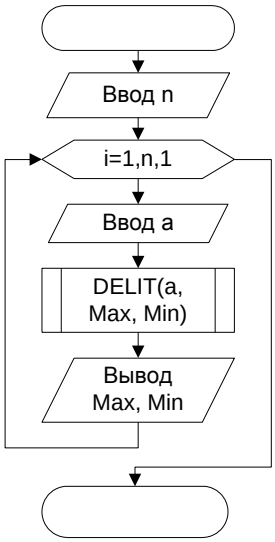
Пример 7.

Дана последовательность из n натуральных чисел. Для каждого числа указать его наименьший и наибольший делитель. Для простого числа – 1 и само число, для остальных это должны быть числа отличные от 1 и самого числа.

Исходные данные: n- количество элементов последовательности. a – элемент последовательности.

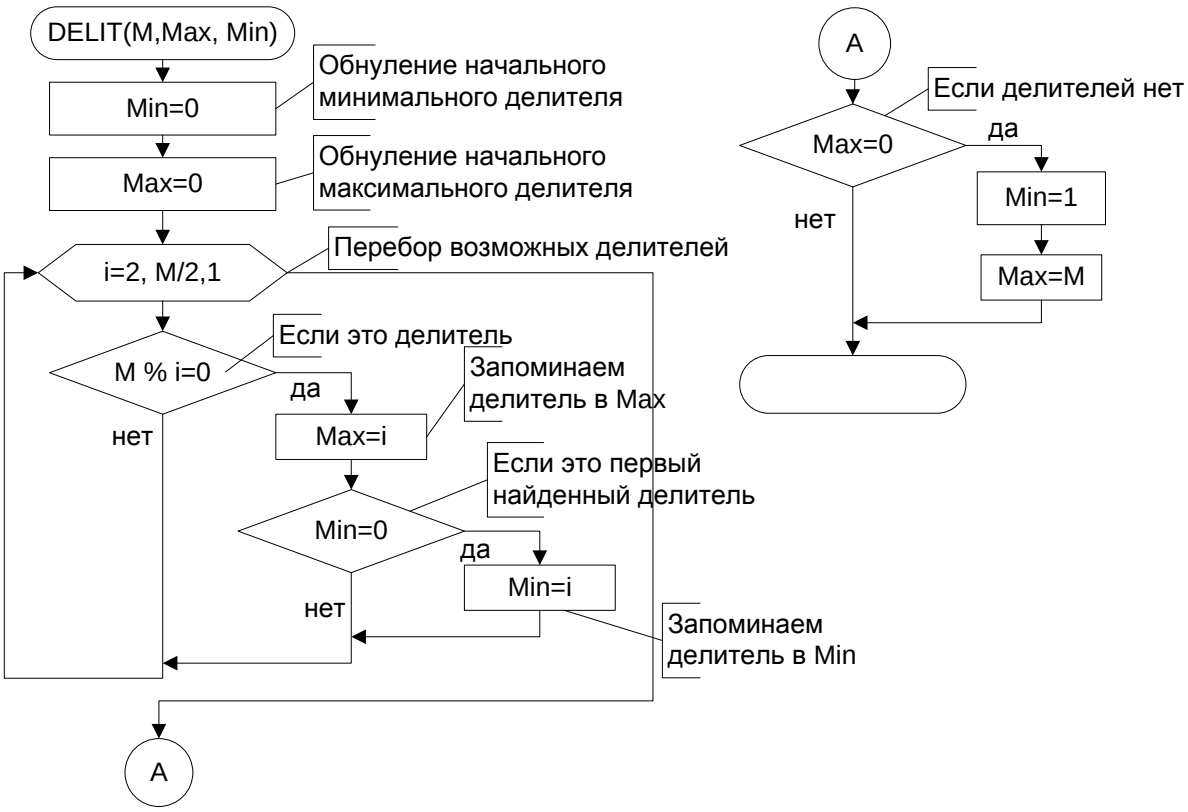
Результат: Max – наибольший делитель, Min – наименьший делитель каждого элемента последовательности.

Нахождение наибольшего и наименьшего делителя выполняется в процедуре DELIT. Void-функция в этом случае используется потому, что в результате получается не одно значение, а два.



Тестовый пример:

при n=5 и
a=6, Max=3, Min-2
a=7, Max=7, Min=1
a=24, Max=12, Min=2
a=15, Max=5, Min=3
a=63, Max=7, Min=3




```
# include <iostream>
# include <conio.h>
# include <clocale>
using namespace std;
```

```
void delit(int m,int& min, int& max);
void main()
{int n,a,min,max;
setlocale(0,"");
 cout<<"n=";cin>>n;
 for(int i=1;i<=n;i++)
 {cout<<"Введите число ";
  cin>>a;
  delit(a,min,max);
  cout<<"мимальный делитель "<<min;
  cout<<" максимальный делитель "<<max<<endl;
  }
 _getch();
}
void delit(int m,int& min, int& max)
{min=0;
 max=0;
 for(int i=2; i<=m/2;i++)
 {
      if (m % i==0) {max=i;
                  if (min==0) min=i;
                                  }
 }
 if (max==0) {min=1;
            max=m;
                    }
 }
```

Пример 8.

Найти все натуральные числа, не превосходящие заданное N, которые делятся на каждую из своих цифр.

Исходные данные: N – целый тип.

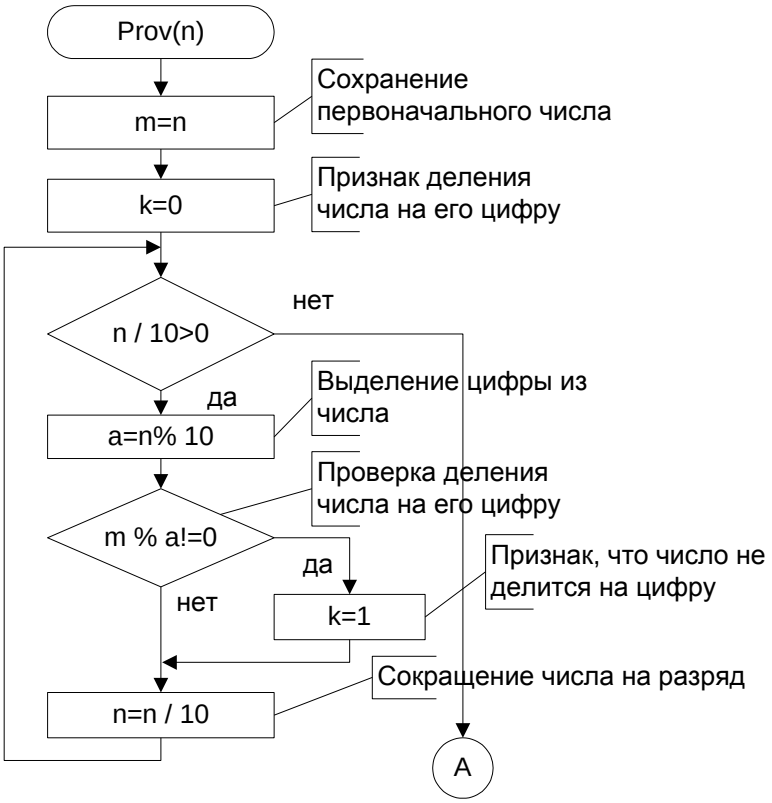
Результат: Вывод чисел, которые делятся на каждую из своих цифр.

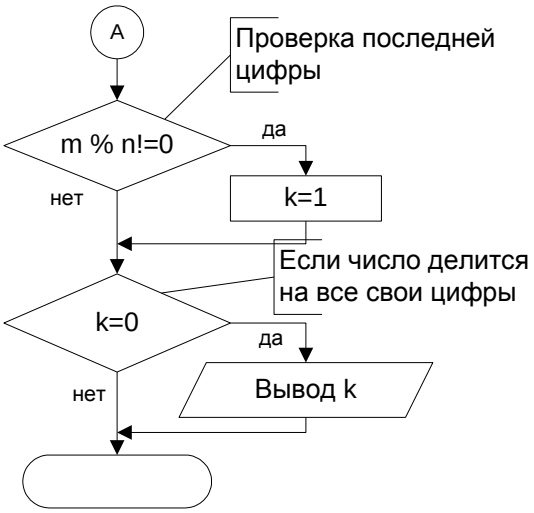
В процедуре Prov проверяется делится ли число на все свои цифры, если делится - данное число выводится на экран. Результатом процедуры является печать чисел, удовлетворяющих заданному условию.

Тестовый пример:

При N=20 выводятся числа: 1,2,3,4,5,6,7,8,9,10,11,12,15,20

Приведен пример блок-схемы процедуры Prov, которая проверяет деление числа на свои цифры.


Prov(n)
m=n
Сохранение первоначального числа
k=0
Признак деления числа на его цифру
n / 10>0
нет
да
a=n% 10
Выделение цифры из числа
m % a!=0
Проверка деления числа на его цифру
да
нет
k=1
Признак, что число не делится на цифру
n=n / 10
Сокращение числа на разряд
A


A
Проверка последней цифры
m % n!=0
да
нет
k=1
Если число делится на все свои цифры
k=0
да
нет
Вывод k